



A HASKELL RUNTIME INTERPRETING SKI COMBINATORS

Matej Rovder

Bachelor's thesis
Faculty of Information Technology
Czech Technical University in Prague
Department of Theoretical Computer Science
Study program: Informatics
Specialisation: Computer Science
Supervisor: RNDr. Tomáš Jakl, Ph.D.
May 14, 2026



Assignment of bachelor's thesis

Title: A Haskell Runtime interpreting SKI combinators
Student: Matej Rovder
Supervisor: RNDr. Tomáš Jakl, Ph.D.
Study program: Informatics
Branch / specialization: Computer Science 2021
Department: Department of Theoretical Computer Science
Validity: until the end of summer semester 2026/2027

Instructions

Study the topic of compilation of functional languages to SKI combinators and also the interpretation of such combinators, according to the literature [2,3].

Implement an interpreter for a subset of combinators, used by the MicroHs compiler [1]. The subset should be sufficient for running basic recursive arithmetic programs and computations with lists and should support basic IO, such as printing on standard output.

The resulting program must be able to accept combinators, using the conventions that MicroHs uses when compiling Haskell programs. Use a programming language which can transpile to JavaScript or at least runs in the browser. The code must be accompanied with a sufficient amount of tests.

References:

- [1] Lennart Augustsson. MicroHs: A Small Compiler for Haskell
<https://dl.acm.org/doi/pdf/10.1145/3677999.3678280>
- [2] D. A. Turner. A New Implementation Technique for Applicative Languages
<https://courses.grainger.illinois.edu/cs522/sp2016/ANewImplementationTechniqueForApplicativeLanguages.pdf>



**FACULTY
OF INFORMATION
TECHNOLOGY
CTU IN PRAGUE**

[3] chapter 16 in the book: Simon L. Peyton Jones. The Implementation of Functional Programming Languages

<https://simon.peytonjones.org/assets/pdfs/slpj-book-1987-2up-searchable.pdf>



Czech Technical University in Prague

Faculty of Information Technology

© 2026 Matej Rovder. All rights reserved.

This thesis is school work as defined by the Copyright Act of the Czech Republic. It has been submitted to the Czech Technical University in Prague, Faculty of Information Technology. The thesis is protected by the Copyright Act and its use, with the exception of free statutory licenses and beyond the scope of the authorizations specified in the Declaration, requires the consent of the author.

Citation of this thesis: Rovder Matej. *A Haskell Runtime interpreting SKI combinators*. Bachelor's thesis. Czech Technical University in Prague, Faculty of Information Technology, 2026.

First of all, I would like to thank my supervisor, RNDr. Tomáš Jakl, Ph.D., for his advice and for exceptional help he provided me. I would also like to thank Ing. Tomáš Pecka for his counsel during my work on this thesis and also for the knowledge he has given me during my studies.

My heartfelt thanks go to my girlfriend Katka for her support and standing at my side. Finally, I want to thank my friends and my family, especially my parents, for the care and help they have given me.

Declaration

I hereby declare that the presented thesis is my own work and that I have cited all sources of information in accordance with the Guideline for adhering to ethical principles when elaborating an academic final thesis.

I acknowledge that my thesis is subject to the rights and obligations stipulated by the Act No. 121/2000 Coll., the Copyright Act, as amended. In accordance with Section 2373(2) of Act No. 89/2012 Coll., the Civil Code, as amended, I hereby grant a non-exclusive authorization (licence) to utilize this thesis, including all computer programs that are part of it or attached to it and all documentation thereof (hereinafter collectively referred to as the “Work”), to any and all persons who wish to use the Work. Such persons are entitled to use the Work in any manner that does not diminish the value of the Work and for any purpose (including use for profit). This authorisation is unlimited in time, territory and quantity.

I declare that I have used AI tools during the preparation and writing of my thesis. I have verified the generated content. I confirm that I am aware that I am fully responsible for the content of the thesis.

In Prague on May 14, 2026

Abstract

This thesis addresses the challenge of running Haskell programs in a web environment. It explores the theory of compiling lambda calculus to SKI combinator expressions and utilizing graph reduction to achieve lazy evaluation. The practical contribution of this thesis is a new runtime system in the TypeScript language for the MicroHs compiler, which compiles Haskell programs to SKI combinator graphs. The resulting runtime provides a lightweight and simple alternative to existing approaches.

Keywords Haskell, MicroHs, functional programming, runtime system, lazy evaluation, lambda calculus, combinators, graph reduction, TypeScript

Abstrakt

Táto práca sa zaoberá možnosťami behu programov v jazyku Haskell vo webovom prostredí. Skúma teóriu prekladu lambda kalkulu na výrazy SKI kombinátorov a použitia grafovej redukcie pre dosiahnutie lenivého vyhodnocovania. Praktickým výsledkom práce je nový runtime systém v jazyku TypeScript pre kompilátor MicroHs, ktorý kompiluje programy z jazyka Haskell na grafy SKI kombinátorov. Výsledný runtime je nenáročnou a jednoduchou alternatívou k existujúcim prístupom.

Kľúčové slová Haskell, MicroHs, funkcionálne programovanie, runtime systém, lenivé vyhodnocovanie, lambda kalkulus, kombinátory, grafová redukcia, TypeScript

Contents

Introduction	2
1 Graph reduction and SKI combinators	4
1.1 Lambda calculus	4
1.2 Reduction strategies in lambda calculus	6
1.2.1 Weak reduction strategies	7
1.2.2 Call-by-need reduction	8
1.3 Graph reduction	8
1.3.1 Graph representation of expressions	9
1.4 SKI combinator calculus	12
1.4.1 Conversion of lambda calculus to SKI combinator calculus	13
1.4.2 Optimization of combinator expressions	17
1.5 Putting everything together	18
1.5.1 Evaluation – runtime system	19
2 Haskell and the MicroHs compiler	21
2.1 Haskell	21
2.2 Existing Haskell compilers	21
2.3 Architecture of MicroHs	22
2.4 MicroHs combinator file	22
2.5 The MicroHs C runtime	26
3 Implementation	27
3.1 Programming languages and tools used	27
3.2 About the codebase	27
3.3 Implementation details	28
3.3.1 Lexing and parsing	28
3.3.2 Representation of the graph	28
3.3.3 Expression evaluation	29
3.3.4 Input/output execution	32
3.3.5 Primitive functions implemented	33
3.4 Measurements	34
3.5 Testing the functionality	36
3.6 Current limitations	36
Conclusion	37

A Usage of the application	38
A.1 Using MicroHs to compile Haskell to combinator code	38
A.2 Usage of the runtime	39
A.3 Running the tests	39
A.4 Lambda expression evaluator	41
Bibliography	42
Contents of the attachment	46

List of Figures

2.1	Architecture of the MicroHs compiler	23
-----	--	----

List of Tables

3.1	Comparison of compiled program size	35
3.2	Measured execution times of the sample programs	36
3.3	Measured memory usage of the sample programs	36

List of Code Listings

1	The types of graph node objects	29
2	The <code>evaluate</code> function code	30
3	Code implementing the reduction rule for the S combinator . .	32

List of Abbreviations

FFI	Foreign function interface
GHC	Glasgow Haskell Compiler
HTML	Hypertext Markup Language
I/O, IO	Input/output
JS	JavaScript
Wasm	WebAssembly
WHNF	Weak head normal form

Introduction

Haskell is a functional programming language, widely used in both industry and academia. The most widely used implementation of Haskell is GHC (Glasgow Haskell Compiler), which compiles Haskell programs into machine code. Its main disadvantage is that it is quite large and difficult to port to different architectures. Other implementations of Haskell, whether interpreters or compilers, were developed in the past, but these have mostly become unmaintained.

In recent years, Lennart Augustsson has started developing MicroHs, a new simpler compiler for Haskell. Its advantage lies in its very small footprint and portability due to its small runtime written in C. MicroHs compiles the Haskell to SKI combinators via lambda calculus and the runtime system executes these combinator programs. Due to the basics of these concepts being easy to grasp, it is possible to create an alternative runtime system for this compiler.

The goal of this work is to implement a runtime system that can be run in a web browser and supports combinator files outputted by MicroHs. As the MicroHs compiler and runtime can already be compiled to WebAssembly using Emscripten, this work aims to make the runtime system code simpler, more understandable and debuggable, with a tiny footprint, and written directly in the TypeScript programming language, rather than compiled to WebAssembly.

Contributions of the thesis

The steps to achieve our goal are as follows:

- Create a parser and compiler of lambda calculus expressions to SKI combinators to showcase the compilation and to be able to test the evaluator before using it with complex programs.
- Write a runtime system (evaluator) for SKI combinator graphs.
- Parse MicroHs combinator output files.
- Make the runtime system compatible with MicroHs-generated output, ensure that I/O (console output) works.
- Write tests to verify the functionality.

Use of AI in this thesis

AI tools were used during the work on this thesis only for grammar and spell checking, code analysis and consultation, configuration of $\text{\LaTeX}$ and of JavaScript tools, and for very minor assistance with the style of writing.

Graph reduction and SKI combinators

1.1 Lambda calculus

Lambda calculus (or λ -calculus) is a formal system of computation, introduced by Alonzo Church. It provides a theoretical foundation for functional programming languages, including Haskell. Lambda calculus is based on performing evaluation steps, called *reductions*, on expressions, called *lambda terms* (or λ -terms).

H. P. Barendregt [1, pp. 22–35] provides a definition of lambda terms and reductions:

► **Definition 1.1** (λ -term). A λ -term is defined inductively:

- a variable (e.g. x) is a λ -term

If M and N are λ -terms and x is any variable, then:

- $(\lambda x.M)$ – a lambda *abstraction*, which represents a function with an argument x and with M as the body – is a λ -term
- $(M N)$ – a lambda *application*, this means that the term M is applied to N – is a λ -term

To make the syntax neater, brackets for applications can be omitted – the operation of application is assumed to be left-associative, i.e. $((xy)z)$ is equivalent to xyz . Brackets in the outermost applications can be omitted as well.

$(\lambda x.M)$ *binds* all free occurrences of x in M . Each variable can be bound only by one abstraction – in case of conflict, it is bound by the closest abstraction. A variable that is not bound is *free*.

Let us define three reduction rules, which describe computation in the lambda calculus. For that we need an auxiliary operation, the *substitution*:

► **Definition 1.2.** We write $M[x/N]$ for substitution, where $M[x/N]$ is the expression obtained from M by replacing every free occurrence of x in M with N .

For example, consider $M[x/u]$ for M shown below. We substitute every free occurrence of x by u , however, there is another abstraction (underlined) in the expression, which binds the variable x .

$$(xy(\underline{\lambda x.x})(\lambda y.x))[x/u]$$

The first occurrence of x is free, so it will be replaced with u . The occurrence of x in the underlined part is bound, it will not be replaced. The last occurrence is not bound, as the abstraction only bounds y , so it will be replaced. The substitution yields:

$$uy(\lambda x.x)(\lambda y.u)$$

► **Definition 1.3** (Reduction rules in lambda calculus).

- **α -conversion:** $(\lambda x.M) \rightarrow (\lambda y.M[x/y])$ if y is not a free variable in M
 α -conversion renames all occurrences of variable x to y
- **β -reduction:** $(\lambda x.M) N \rightarrow M[x/N]$ if no variable free in N becomes bound in $M[x/N]$
 β -reduction is the main computation rule of the lambda calculus, we can say it is the operation which “executes” an application
- **η -conversion:** $(\lambda x.Mx) \rightarrow M$ if x is not a free variable in M
 η -conversion (or η -reduction) is a rule that optimizes an expression, it does not affect the result of computation, because $(\lambda x.Mx)$ and M are computationally equivalent

An α -conversion is usually done when a β -reduction is desired but impossible, because a free variable would become bound. The α -conversion (or multiple conversions) renames one (or some) of the conflicting variables to avoid the name conflict.

A term is in *normal form* if no β or η reduction rule can be applied to any of its α -conversions. The goal of evaluating a λ -term is to find its normal form, which may be impossible, depending on the term and the order of reduction steps taken.

1.2 Reduction strategies in lambda calculus

A *reduction strategy*¹ in the context of lambda calculus determines the order of reductions performed, more specifically, which reducible expression (*redex*) to choose for β -reduction in each step. Following B. C. Pierce [2, pp. 55–58], we explain the most important reduction strategies:

Normal order reduces the leftmost outermost redex first. That means, when reading the expression from left to right, we reduce the first application not inside a λ -abstraction we come across (outermost). If there is no such, we look for redexes inside abstractions, then in abstractions that are inside abstractions etc. Let us demonstrate it on a rather arbitrary expression:

$$(\lambda x. (\lambda y. \underline{xx}))((\lambda a. w)b)((\lambda a. a)c)$$

The leftmost outermost redex is the underlined application, we apply the β -reduction rule and substitute the argument for x .

$$\rightarrow (\lambda y. ((\lambda a. w)b)((\lambda a. w)b))((\lambda a. a)c)$$

Observe that the argument for x now occurs twice in the expression – to reach the normal form, we will have to evaluate it both times. The next redex to reduce is the outer one, we substitute $((\lambda a. a)c)$ for y – which is not used in the body of the abstraction, hence we obtain the following:

$$\begin{aligned} &\rightarrow ((\lambda a. w)b)((\lambda a. w)b) \\ &\rightarrow w((\lambda a. w)b) \\ &\rightarrow ww \end{aligned}$$

Applicative order [3] reduction reduces the leftmost innermost² redex first, which means that before doing a β -reduction, the argument gets reduced first.

$$(\lambda x. (\lambda y. \underline{xx}))(\underline{((\lambda a. w)b}})((\lambda a. a)c)$$

The leftmost innermost redex is in the argument, as underlined, we apply the β -reduction rule and substitute b for a , which does not appear in the body – the argument gets reduced to w .

$$\rightarrow (\lambda x. (\lambda y. \underline{xx}))w((\lambda a. a)c)$$

The argument of the leftmost application is now in its normal form (w) and the evaluation continues with β -reduction of the underlined redex.

$$\rightarrow (\lambda y. ww)(\underline{((\lambda a. a)c}})$$

¹Often referred to as evaluation strategy [2, p. 56], but the name “reduction strategy” is more specific for lambda calculus.

²More precisely, this is the leftmost of the innermost redexes.

The next application to reduce is the other argument, we substitute c for a .

$$\begin{array}{c} \rightarrow (\lambda y.ww)c \\ \hline \rightarrow ww \end{array}$$

We can see that neither strategy is perfect. Normal order reduction had us copy the argument $((\lambda a.w)b)$ by substituting it for x , having to perform the same evaluation steps on it twice. Applicative order reduction, on the other hand, made us evaluate an argument that was not even used in the function body.

Also, there are expressions for which the applicative order reduction cannot reach the normal form (the reduction can get “stuck” evaluating an argument that is not required).

For example, the term $(\lambda x.y)((\lambda z.zz)(\lambda z.zz))$ gets reduced to its normal form y in just one step of normal order. Applicative order however chooses to reduce the application $((\lambda z.zz)(\lambda z.zz))$, which gets reduced to itself – and so the whole term gets reduced to itself, never reaching a normal form.

Normal order reduction, on the other hand, is proven to lead to the normal form whenever the normal form exists – see Normalization theorem, [1, pp. 326–327]. This is a corollary of the Church-Rosser theorem [1, pp. 282–283]. Its another corollary is that a term can have at most one normal form up to α -equivalence.

1.2.1 Weak reduction strategies

The majority of programming languages offer no equivalent to reduction inside lambda abstractions – that corresponds to evaluation inside of function bodies, whether or not is the function called. The weak variants of reduction strategies in lambda calculus are devised to reflect on that.

The *call-by-name* and *call-by-value* weak reduction strategies are weak versions of the normal and applicative strategies, respectively. That is, they behave as these two strategies, except that they do not allow reduction inside abstractions. The *call-by-name* strategy reduces the leftmost outermost redex not inside a lambda abstraction first and disallows evaluation of application arguments. The *call-by-value* strategy reduces the leftmost innermost redex not inside an abstraction. [4]

The *call-by-name* reduction strategy reduces the expression to *weak head normal form* [4, 5, p. 198].

► **Definition 1.4** (Weak head normal form). A lambda expression is in weak head normal form (WHNF) if and only if it is of the form $\lambda x.e$ or $x e_1 e_2 \dots e_n$ where $n \geq 0$, x is a variable. In case of lambda calculus with constants for built-in functions, also $(F e_1 e_2 \dots e_n)$ is in WHNF if F has arity m and $n \leq m$.

1.2.2 Call-by-need reduction

Most programming languages use call-by-value evaluation strategy, whereas Haskell [6] uses *call-by-need*, in the context of programming languages better known as *lazy evaluation*. The *call-by-need* reduction strategy is an optimized version of call-by-name. It uses sharing to avoid evaluating the same expression multiple times [2, p. 57].

For example, in this expression:

$$(\lambda x. (\lambda y. \underline{xx}))((\lambda a. w)b)((\lambda a. a)c),$$

we reduce the leftmost outermost redex (underlined), just like in normal order. However, the argument becomes a shared subexpression, which will be referenced by x . The evaluator “stores” that $x = ((\lambda a. w)b)$ and proceeds with:

$$\rightarrow (\lambda y. \underline{xx})((\lambda a. a)c)$$

The evaluation continues similarly to normal order, $((\lambda a. a)c)$ is to be substituted for y , which however has no occurrence. We obtain:

$$\rightarrow \mathbf{xx}$$

Finally, we have to evaluate $\mathbf{x}$. We know that $\mathbf{x} = ((\lambda a. w)b)$, which gets reduced to w . This yields us:

$$\rightarrow ww$$

For more details on how this is implemented, see Section 1.3 and the chapter 12 in [5].

1.3 Graph reduction

Graph reduction is a technique of evaluating an expression as an implementation of the lazy evaluation strategy.

The main idea of graph reduction is that due to the expression being represented as a graph, there can be multiple edges leading into the same node – as opposed to reduction on trees. This enables sharing because it lets the same subexpression have multiple occurrences which share the same subgraph. Then, as a result, if one of these occurrences has to be evaluated, the shared subgraph gets evaluated and other occurrences of the subexpression do not need to be evaluated again.

Graph reduction was first described by C. P. Wadsworth [7] in his doctoral thesis about call-by-need reduction of lambda calculus. An expression (or a program) can be represented in the form of a graph, and the evaluation of the expression (or the execution of the program) consists of steps called *reductions*, which transform this graph according to *reduction rules*.

P. Henderson and J. H. Morris [8] were the first to implement a lazy evaluator for a programming language using graph reduction. Many various implementations of lazy evaluators are also built on this principle, such as Turner's reduction machine [9] (see also Section 1.5), which was used by Turner to implement Miranda [10], a programming language which influenced Haskell [11, p. 12-11]. Another example of a graph reduction machine is the more recent G-machine, an improved version of which is used in GHC [11, pp. 12-28–12-29].

1.3.1 Graph representation of expressions

Let us take a closer look at how graph reduction can be used to evaluate lambda expressions. Execution of programs compiled to lambda calculus or SKI combinators (see Section 1.4) works very similarly.

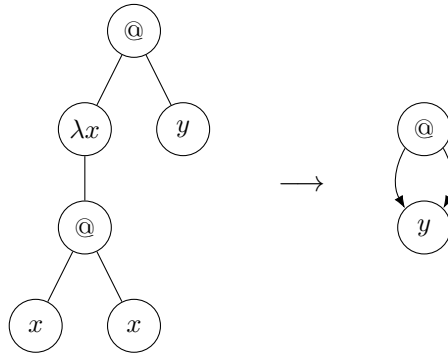
The graph representation of an expression has directed edges – in our diagrams below, if an edge has no explicit direction, it can be assumed to point downward. Again, multiple edges can lead into the same node if the node (or its subtree) is shared. The nodes in this graph are of these types:

- an application ($@$), which has two edges leading from it to represent the left-hand and right-hand side of the application
- an abstraction (λ), which has one edge to the body of the abstraction, while the abstraction variable name is in the node itself
- a node representing a variable or another data object (a number, built-in function name³ or a combinator – we define combinators in Section 1.4), which has no outgoing edges

Although the graph representation might contain cycles, we will refer to the graphs on the left-hand and right-hand side of an application as *subtrees* and we will intuitively understand their top-level nodes as their *root*.

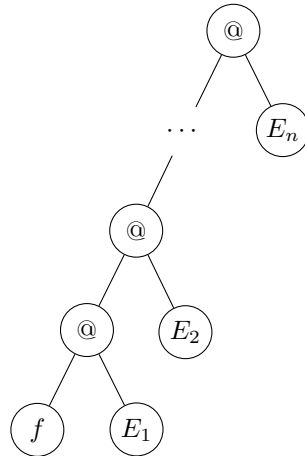
As an example, consider the expression $(\lambda x.xx)y$. The tree on the left-hand side of the diagram below is obtained after parsing this expression and the graph on the right-hand side is obtained by performing a reduction to yy . In the resulting expression, the node y is shared, which introduces a cycle:

³A special, primitive function or operation implemented by the evaluation machine – arithmetic operations for example. In the graph, it can be represented by a node with the name of the function.



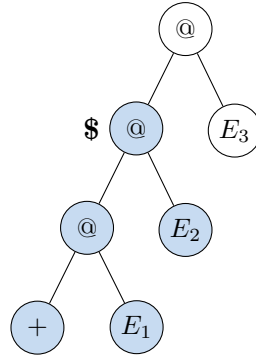
The reduction follows the principles of call-by-need reduction, explained in Section 1.2.2. Peyton Jones [5, p. 201] gives a good explanation on which redex to reduce (how to find the leftmost redex):

The expression to reduce can only be of form $f E_1 E_2 \dots E_n$, the graph of which looks like this (due to left associativity):



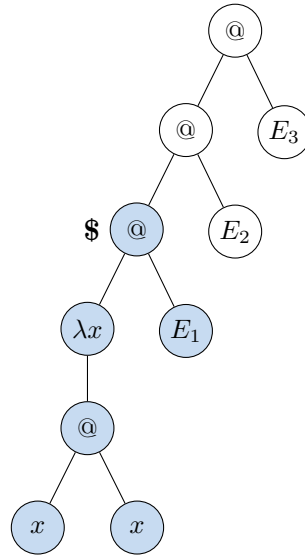
- If f is a variable (or other data object), the expression is in WHNF and the evaluation is done.
- If f is a built-in function with arity k , we must first check that there are enough arguments available, i.e. $n \geq k$. If so, $(f E_1 \dots E_k)$ is the outermost redex, which will be selected by normal order. Otherwise, the expression is in WHNF.

For example, in this graph (+ being a function of two arguments) the redex is $(+ E_1 E_2)$ – it is coloured and the root of its subtree is marked with \$:



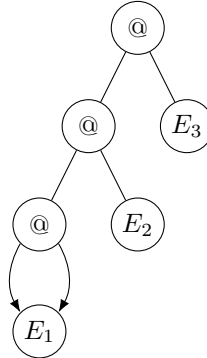
- If f is a lambda abstraction⁴, the redex to reduce is $(f E_1)$ provided that the argument E_1 is available. Otherwise, the expression is in WHNF.

In the following example, the redex to reduce is $((\lambda x.x x)E_1)$:



The redex was reduced to $(E_1 E_1)$ – after the reduction, the whole expression is $(E_1 E_1 E_2 E_3)$. E_1 became a shared node, used twice in the graph:

⁴It is important to note that when applying lambda abstractions, the substitutions must be made within a new copy of the body of the abstraction, as the original body can be used multiple times in the case that it is shared [5, pp. 209–210].



1.4 SKI combinator calculus

A naive implementation of lambda calculus using graph reduction would still be quite inefficient: substitution of each argument into the body of an abstraction would require a recursive tree-walk over the body (see [5, pp. 209–210]). SKI combinator calculus allows us to remove abstractions and bound variables and thus streamline the evaluation.

The main ideas come from combinatory logic, as described by Schönfinkel (1924) [12] and later H. B. Curry [13]. Combinatory logic was introduced as a means to eliminate the need for quantified variables from mathematical logic and predates lambda calculus, introduced by A. Church [14] in 1932.

The SKI combinatory calculus defines and uses various constants called *combinators*. Each combinator has a defined reduction rule: a combinator $\mathbb{C}$ of the arity n has a reduction rule $\mathbb{C} x_1 x_2 \dots x_n \rightarrow E$, where E is an expression containing only combinators and variables from the set $\{x_1, x_2, \dots, x_n\}$.

Schönfinkel has defined a larger set of combinators, but the combinators **S** and **K** are sufficient to express all of the other combinators. The **I** (identity) combinator, although not necessary, is also often placed with **S** and **K** in the “basic set” of combinators – that is why the computational model based on Schönfinkel’s combinatory logic system is called SKI⁵ combinator calculus.

► **Definition 1.5** (SKI combinators). The **S**, **K** and **I** combinators are defined by the following reduction rules:

$$\mathbf{S} \ x \ y \ z \rightarrow x \ z \ (y \ z)$$

$$\mathbf{K} \ x \ y \rightarrow x$$

$$\mathbf{I} \ x \rightarrow x$$

⁵As **I** is redundant, the name SK is also often used.

1.4.1 Conversion of lambda calculus to SKI combinator calculus

Many authors, including S. L. Peyton Jones [5, pp. 260–264] and D. A. Turner [9, p. 34], describe a set of rules for conversion of lambda expressions (or functional programs) to SKI combinator calculus⁶. Combinatory logic has no notion of abstraction or variable substitution, the conversion removes all bound variables from the expression. The evaluation of such expression is then very simple, the evaluator only applies the reduction rules from the definition of the combinators (Definition 1.5) while possible.

We list the rules for conversion as presented by Peyton Jones [5, pp. 264, 273], in the form of function **Compile** which compiles a lambda expression into SKI combinators. We will also use the functions **Abs** for abstraction and **Opt** for optimization, these two functions are just auxiliary.

► **Definition 1.6** (Rules for conversion of lambda expressions to SKI).

Compile $[e]$ compiles e to SKI combinators:

$$\begin{aligned}\mathbf{Compile}[e_1 e_2] &\Rightarrow \mathbf{Compile}[e_1] \mathbf{Compile}[e_2] \\ \mathbf{Compile}[\lambda x.e] &\Rightarrow \mathbf{Abs } x [\mathbf{Compile}[e]] \\ \mathbf{Compile}[cv] &\Rightarrow cv\end{aligned}$$

Abs $x [f]$ abstracts the variable x from f :

$$\begin{aligned}\mathbf{Abs } x [f_1 f_2] &\Rightarrow \mathbf{Opt}[\mathbf{S } (\mathbf{Abs } x [f_1]) (\mathbf{Abs } x [f_2])] \\ \mathbf{Abs } x [x] &\Rightarrow \mathbf{I} \\ \mathbf{Abs } x [cv] &\Rightarrow \mathbf{K } cv\end{aligned}$$

where

- cv is a constant (including combinators and other data types) or a variable other than x
- e, e_1, e_2 are any expressions
- f, f_1, f_2 are expressions with no inner lambda abstractions⁷
- **Opt** is an optimization of a combinator expression, which we will explain in the next section (Section 1.4.2)

⁶See that although Peyton Jones' rules deal with conversion of lambda calculus and Turner's paper is about implementing a programming language named SASL, the rules and the process of conversion are virtually the same.

⁷Notice that we apply **Compile** to the body of a lambda abstraction before applying **Abs**, so that **Abs** does not have to deal with lambda abstractions.

For now (in the next example), we only allow the simple optimization rule $\mathbf{Opt} [\mathbf{S} (\mathbf{K} p) \mathbf{I}] = p$ to be used, as it is equivalent to performing an η -conversion just before doing an abstraction. For expressions not matching the left-hand side of this rule, $\mathbf{Opt}$ will behave as an identity function. More optimization rules are listed in Definition 1.8.

As an example, let us compile the lambda expression $(\lambda x.(\lambda y.xx))$ into combinators. To keep the expressions more concise, we will explicitly write $\mathbf{Opt}$ only where an optimization will take place.

$$\begin{aligned}
& \mathbf{Compile}[(\lambda x.(\lambda y.xx))] \\
& \Rightarrow \mathbf{Abs} x [\mathbf{Compile}[(\lambda y.xx)]] \\
& \Rightarrow \mathbf{Abs} x [\mathbf{Abs} y [\mathbf{Compile}[xx]]] \\
& \Rightarrow \mathbf{Abs} x [\mathbf{Abs} y [xx]] \\
& \Rightarrow \mathbf{Abs} x [\mathbf{S}(\mathbf{Abs} y [x])(\mathbf{Abs} y [x])] \\
& \Rightarrow \mathbf{Abs} x [\mathbf{S}(\mathbf{K}x)(\mathbf{K}x)]
\end{aligned}$$

This matches the rule $\mathbf{Abs} x [f_1 f_2]$ – as the expressions associate to the left, $\mathbf{S}(\mathbf{K}x)$ will bind to f_1 and $(\mathbf{K}x)$ will bind to f_2 .

$$\Rightarrow \mathbf{S}(\mathbf{Abs} x [\mathbf{S}(\mathbf{K}x)])(\mathbf{Abs} x [\mathbf{K}x])$$

We could do an η -conversion on the abstraction $\mathbf{Abs} x [\mathbf{K}x]$ and get $\mathbf{K}$, but we will let the optimization rule take care of that.

$$\begin{aligned}
& \Rightarrow \mathbf{S}(\mathbf{S}(\mathbf{Abs} x [\mathbf{S}]) (\mathbf{Abs} x [\mathbf{K}x])) (\mathbf{Opt} [\mathbf{S}(\mathbf{Abs} x [\mathbf{K}]) (\mathbf{Abs} x [x])]) \\
& \Rightarrow \mathbf{S}(\mathbf{S}(\mathbf{KS})) (\mathbf{Opt} [\mathbf{S}(\mathbf{Abs} x [\mathbf{K}]) (\mathbf{Abs} x [x])]) (\mathbf{Opt} [\mathbf{S}(\mathbf{KK})\mathbf{I}])
\end{aligned}$$

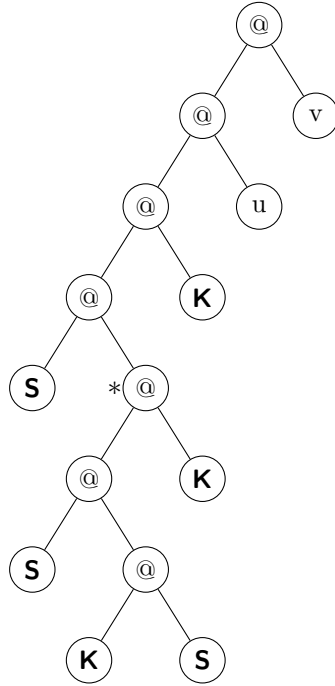
Now, the subexpression $\mathbf{S}(\mathbf{KK})\mathbf{I}$ matches the left-hand side of the optimization rule, so we can reduce it to just $\mathbf{K}$. There is another occurrence of this subexpression:

$$\begin{aligned}
& \Rightarrow \mathbf{S}(\mathbf{S}(\mathbf{KS})) (\mathbf{Opt} [\mathbf{S}(\mathbf{KK})\mathbf{I}]) \mathbf{K} \\
& \Rightarrow \mathbf{S}(\mathbf{S}(\mathbf{KS})\mathbf{K})\mathbf{K}
\end{aligned}$$

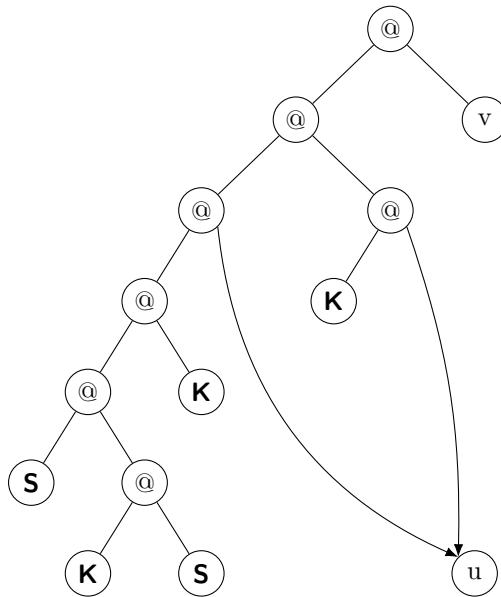
Based on Section 1.3.1, we can intuitively understand how evaluation of this expression will proceed and we will demonstrate it on the example below. For a more rigorous description of an evaluation algorithm, see Section 1.5.

The next diagram is the representation of the previous term as a graph. We apply the term on two arguments, u and v , to showcase the evaluation.

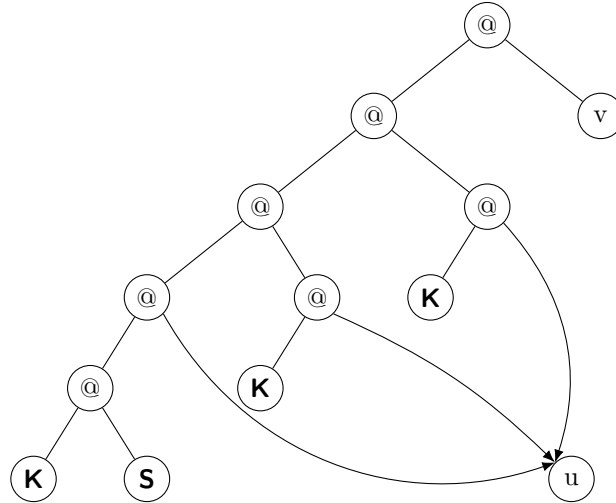
The topmost redex is $\mathbf{S} * \mathbf{K} u$, where $*$ represents the whole subexpression whose root is marked with $*$ in the graph:



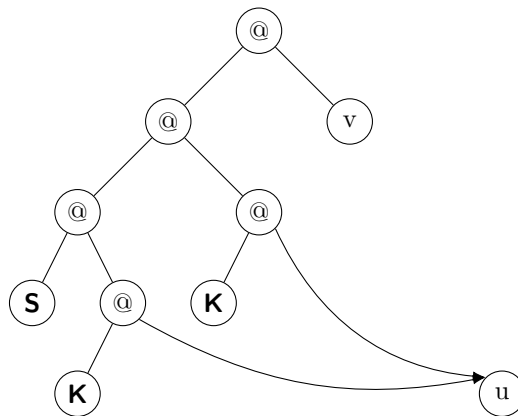
After the $\mathbf{S}$ reduction, the node u is used twice in the graph. Instead of being copied, it is now shared among the graph. The next redex to be reduced is $(\mathbf{S} (\mathbf{K} \mathbf{S}) \mathbf{K} u)$:



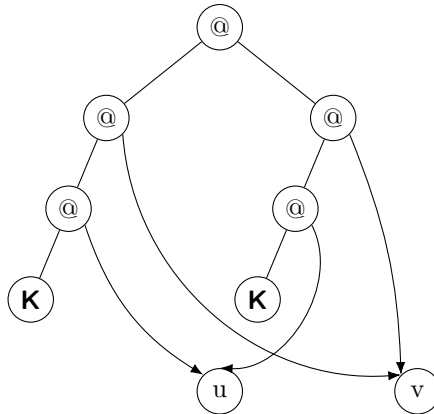
The reduction of $(\mathbf{K} \mathbf{S} u)$ follows:



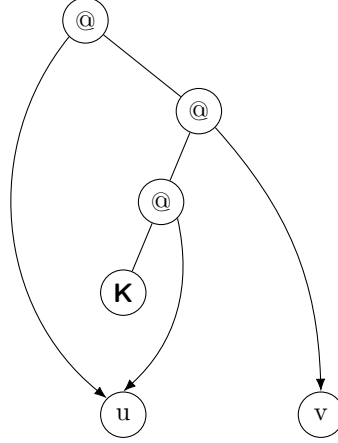
Now, the redex is the whole graph:



The node v has just become shared as well. The next redex to reduce is $(\mathbf{K} u v)$:



The evaluation has reached the WHNF of the expression and therefore it is finished: $u(\mathbf{K}uv)$. This is the graph of the expression:



If we were to continue evaluation inside the application on the right, the expression would reduce to its normal form uu – this is also what the evaluation of the original lambda expression applied to two arguments: $(\lambda x.(\lambda y.xx))uv$, would yield.

1.4.2 Optimization of combinator expressions

After the conversion, the resulting expressions, namely those in the form $\mathbf{S} e_1 e_2$, can then be optimized, so that the evaluation takes fewer reductions. For this, several new combinators are introduced (Definition 1.7). Turner [9] introduces four rules and defines new combinators $\mathbf{B}$ and $\mathbf{C}$, then in his later paper [15], he adds $\mathbf{S}'$, $\mathbf{B}'$, and $\mathbf{C}'$. Peyton Jones [5, pp. 267–273] uses these combinators except $\mathbf{B}'$ ⁸ and also lists the $\mathbf{B}^*$ combinator (not used in MicroHs [16]), defined by Mark Sheevel. The optimization rules, as per Peyton Jones [5, p. 273], are listed in Definition 1.8. The choice of optimizations affects the specific WHNF reached during evaluation.

► **Definition 1.7** (Additional combinators listed by Peyton Jones or Turner).

$$\begin{aligned}
 \mathbf{B} \ x \ y \ z &\rightarrow x \ (y \ z) \\
 \mathbf{C} \ x \ y \ z &\rightarrow x \ z \ y \\
 \mathbf{S}' \ x \ y \ z \ w &\rightarrow x \ (y \ w) \ (z \ w) \\
 \mathbf{C}' \ x \ y \ z \ w &\rightarrow x \ (y \ w) \ z \\
 \mathbf{B}^* \ x \ y \ z \ w &\rightarrow x \ (y \ (z \ w)) \\
 \mathbf{B}' \ x \ y \ z \ w &\rightarrow x \ y \ (z \ w)
 \end{aligned}$$

⁸According to Peyton Jones, usage of $\mathbf{B}'$ combinator degrades performance, but it is nonetheless used by Augustsson [16].

► **Definition 1.8** (Rules for the optimization of SKI expressions).

Opt $[e]$ optimizes the expression e :

$$\begin{aligned}
\text{Opt } [\mathbf{S} (\mathbf{K} p) (\mathbf{K} q)] &\Rightarrow \mathbf{K} (p q) \\
\text{Opt } [\mathbf{S} (\mathbf{K} p) \mathbf{I}] &\Rightarrow p \\
\text{Opt } [\mathbf{S} (\mathbf{K} p) (\mathbf{B} q r)] &\Rightarrow \mathbf{B}^* p q r \\
\text{Opt } [\mathbf{S} (\mathbf{K} p) q] &\Rightarrow \mathbf{B} p q \\
\text{Opt } [\mathbf{S} (\mathbf{B} p q) (\mathbf{K} r)] &\Rightarrow \mathbf{C}' p q r \\
\text{Opt } [\mathbf{S} p (\mathbf{K} q)] &\Rightarrow \mathbf{C} p q \\
\text{Opt } [\mathbf{S} (\mathbf{B} p q) r] &\Rightarrow \mathbf{S}' p q r \\
\text{Opt } [e] &\Rightarrow e \quad \text{otherwise}
\end{aligned}$$

For the purposes of MicroHs, Augustsson defines several other combinators, which he has empirically proven to improve performance. Additionally, the combinator $\mathbf{Y}$ is used for local recursion and unlike in lambda calculus, it creates a cycle instead of copying [16]. These combinators are used in MicroHs⁹ and the reduction rules for them are implemented in our runtime system. The following combinators are used by Augustsson and are not listed in the previous definitions:

► **Definition 1.9** (Additional combinators used in MicroHs [16]).

$$\begin{aligned}
\mathbf{A} x y &\rightarrow y \\
\mathbf{U} x y &\rightarrow y x \\
\mathbf{Y} x &\rightarrow x (\mathbf{Y} x) \\
\mathbf{Z} x y z &\rightarrow x y \\
\mathbf{P} x y z &\rightarrow z x y \\
\mathbf{R} x y z &\rightarrow y z x \\
\mathbf{O} x y z w &\rightarrow w x y \\
\mathbf{K2} x y z &\rightarrow x \\
\mathbf{K3} x y z w &\rightarrow x \\
\mathbf{K4} x y z w v &\rightarrow x \\
\mathbf{C'B} x y z w &\rightarrow x z (y w)
\end{aligned}$$

1.5 Putting everything together

Turner [9] described a technique to implement *applicative* languages (a group of functional languages) using SKI combinators and graph reduction. This technique allows for lazy evaluation in a simple and efficient way. It was used

⁹For more information on the optimization rules used in MicroHs, see the MicroHs source code [17], path: `src/MicroHs/Abstract.hs`.

by Turner to implement the SASL programming language [9] and the more successful Miranda [10].

The programs are compiled according to the rules in Section 1.4. The resulting SKI expressions take the form of graphs, as described in Section 1.3.1.

1.5.1 Evaluation – runtime system

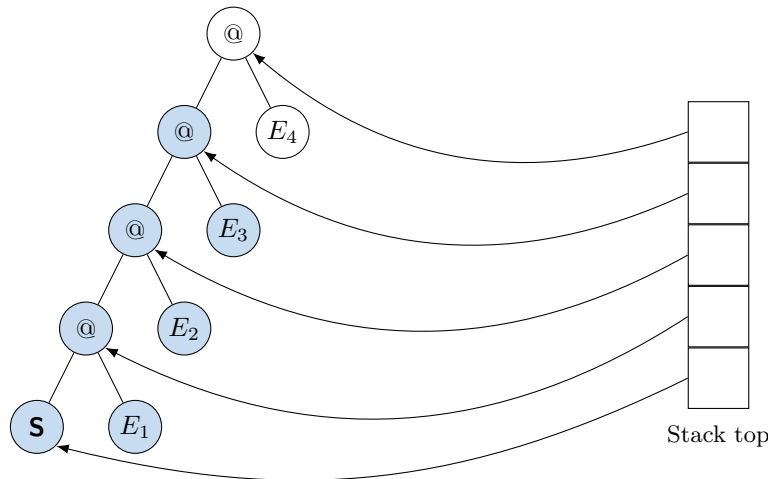
Turner’s runtime system, or the *S–K reduction machine* [9, from p. 39], as he named it, transforms the graph structure according to the rules from Definitions 1.5, 1.7, and 1.9. The node matching the left side of a rule is replaced with the corresponding right side. The concrete implementation of an evaluation machine should also support some primitive built-in functions, such as arithmetic operations, comparisons and conditionals.

To ensure lazy evaluation strategy, the reductions must be performed in the correct order, i.e. the leftmost redex is reduced (see Sections 1.2 and 1.2.2).

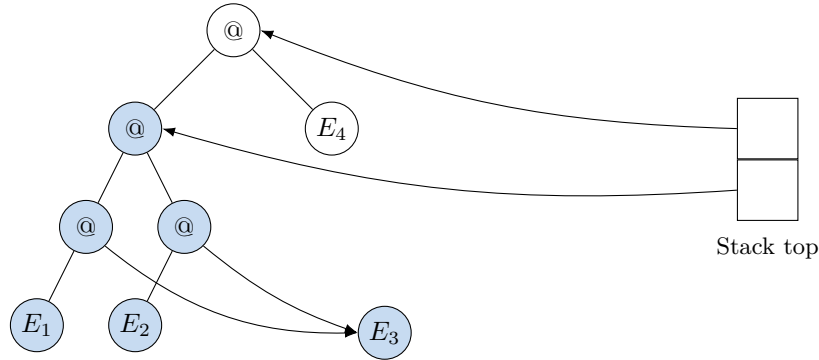
To reach the leftmost redex, we keep a stack of ancestors. For as long as the expression on the top of the stack is an application, we take its left subtree and push it onto the stack.

When we finally get an atom on the top, if it is a combinator, we apply the appropriate reduction rule. We begin by removing the combinator from the top of the stack. The i -th argument of the combinator is the right subtree of the application node i places on the stack behind the combinator. That means we pop N nodes from the stack, where N is the arity of the combinator, keeping their right subtrees, which we use while applying the reduction rule.

In the next example, there is a ternary combinator **S** on the top, therefore its arguments are the right subtrees of the three following nodes on the stack. The top-level redex is $(\mathbf{S} E_1 E_2 E_3)$, highlighted in the graph:



The expression $(\mathbf{S} E_1 E_2 E_3)$ is reduced to $((E_1 E_3) (E_2 E_3))$ and the new expression (highlighted) replaces the subtree of the original redex:



If the atom at the top is, say, an arithmetic operation, e.g. addition, its arguments have to be evaluated by recursively calling the evaluation function on them before they are added.

After a reduction rule is applied (or a primitive function is executed), the result is pushed onto the stack. As this evaluation system is supposed to transform the graph, the left subtree pointer of the node before is set to point to the resulting node. This ensures that the expression graph is valid after each evaluation step, so that if a recursive call of the evaluation function reaches the graph, it is not just in a valid state, but it has already been partially evaluated, and the recursive call does not have to perform the same evaluation steps.

The sequence of reductions continues until there is only one item left in the stack, which is a number or a value that can be returned as the result of the evaluation. If such a value is on the top of the stack, but there are still applications in the stack – the value, which is neither a combinator nor a function, is applied to something – this is an invalid state caused probably by an invalid program. Similarly, if a combinator or a function is found to not have enough arguments on the stack, or a function is applied to a value of an unsupported type, the evaluation cannot continue nor return a result other than an error state.

Memory management As the programs are represented by cyclic graphs, simple reference counting is not a viable approach to memory management. It may therefore be favorable to choose an implementation language with built-in garbage collection, such as TypeScript/JavaScript, used by this work. In contrast, the MicroHs C runtime uses a custom garbage collector.

Haskell and the MicroHs compiler

2.1 Haskell

Haskell [6] is a purely functional programming language. As such, it is based on the lambda calculus (Section 1.1). It supports higher-order functions, algebraic data types, pattern matching, type classes, modules, etc.

2.2 Existing Haskell compilers

GHC (Glasgow Haskell Compiler) [18] is the mainstream [19] Haskell compiler, but it is quite heavy – a basic installation size of GHC 9.6.7 with required libraries is almost 2 gigabytes, making it unsuitable to run client-side on the web. Recently, a JavaScript backend [20] has been added to the GHC compiler¹⁰. Using the JavaScript backend, GHC can compile Haskell code to JavaScript. The resulting JS code is however unreasonably large – several megabytes even for simple programs (more details can be found in Section 3.4), the reason for this is that the GHC runtime system is bundled with the code.

MicroHs [17], a new minimalistic Haskell compiler, supports being compiled to JavaScript and WebAssembly using Emscripten. Again, this is accomplished by compiling the whole C runtime into JS/Wasm, bundled with the Haskell code compiled into the format supported by the runtime (combinator graph). As before, this has a number of disadvantages, namely, the C runtime brings its own garbage collector when JS already has one. Another disadvantage is that any communication with the browser has to be done using an extra hand-written JS layer. Lastly, the resulting code is difficult to read and understand, as it is automatically generated, prioritizing efficiency.

¹⁰Do not confuse this new GHC JavaScript backend with the GHCJS [21] project, which is out of development and does not support the latest GHC versions.

As stated previously, this work creates a replacement web-native runtime system for MicroHs, using MicroHs only as a compiler (the next section gives a more in-depth overview of the architecture of MicroHs). This will get us a runtime with human-readable code, simple enough to understand and debug, which is also small in size.

2.3 Architecture of MicroHs

Although the inner workings of MicroHs are not the focus of this work, we give a short overview to explain which part of MicroHs we aim to replace.

MicroHs consists of two main parts: the compiler itself (Figure 2.1) and the runtime system. The first stage of compilation is lexing and parsing, followed by type checking. The next two stages, desugaring and encoding data types, are of more interest, as they translate the program into lambda calculus including the necessary literals and primitives. Following are the optimization stage and the code generation stage, which compiles the lambda calculus representation into SKI combinator expressions (in the form of a graph) and optimizes these expressions (Sections 1.4.1 and 1.4.2) [16].

The combinator graph representation of the program is serialized into text form and can be either saved as a standalone `.comb` file or bundled as a big C array of characters with the default runtime to create an executable. In Section 2.4, we give more information about the output format.

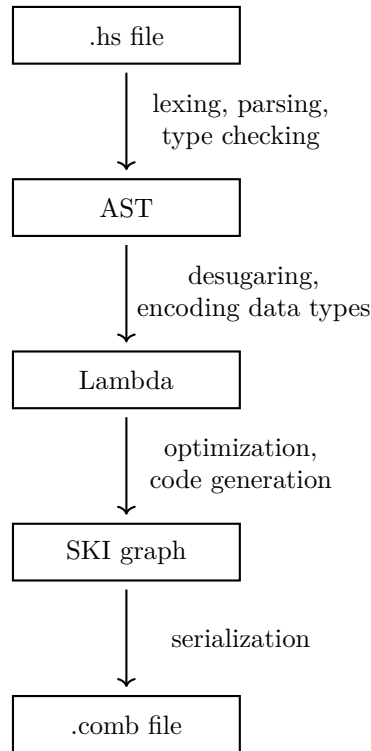
The default MicroHs runtime is written in C. It first parses the combinator graph representation of the program, loading the graph into memory, and then it runs the program by reducing the graph using the reduction steps from Definitions 1.5, 1.7, and 1.9. This is the part that we are replacing with our TypeScript implementation.

2.4 MicroHs combinator file

The first step in implementing a runtime system for MicroHs is to parse its combinator (`.comb`) output. The format is pretty simple, although it is not formally documented, it is easy enough to understand its structure from the `parse` function in the C runtime code [17].

The `.comb` file contains a serialized form of the combinator graph representing the compiled program. While parsing the file, the parser creates the graph nodes and builds the graph. The file format allows for numeric labels to be assigned to some expressions, which is required to represent a graph with shared subexpressions. We will refer to such expressions with an assigned label as numbered or shared expressions.

The header of the file has the version of the format on the first line and the count of numbered expressions on the second line.



■ **Figure 2.1** Architecture of the MicroHs compiler

The body of the file holds the combinator graph in a postfix form – requiring only a simple parser with a stack, which holds graph nodes of various types. If the parser encounters the character @ (the at sign represents application), it pops two nodes from the stack, makes an application node – the expression at the second-to-top position is applied to the one on the top – and pushes this application node onto the stack.

The majority of tokens in the combinator file represent some kind of a node in the graph. If the parser encounters such a token, it will create the corresponding node and push it onto the stack.

- A sequence of characters not matching any other rule represents a combinator (if a combinator with such name exists) or a call to a function implemented by the runtime system.
- A hash symbol (#) indicates that the following is an integer literal
- A quotation mark (") indicates the start of a character string. MicroHs uses an unusual encoding¹¹ for characters outside the printable ASCII range, but it is simple enough to decode it.

¹¹For further details about this encoding, see the MicroHs source code [17], path: `src/MicroHs/ExpPrint.hs`. Additionally, a decoder is implemented both in our lexer and the MicroHs runtime (`src/runtime/eval.c` in [17]).

- A caret (^) means that the following is an FFI (foreign function interface) call. As FFI is not supported in this work, it is replaced with a simple function call, then if the function is unknown, a runtime error is thrown if it is to be called.
- An underscore _ is followed by a number (label) and it represents a reference to a shared expression with that label.
- A definition of a shared expression is denoted by a colon (:) followed by a number (label) and it creates an entry in the table of shared expressions. In this entry, the expression on the top of the stack is assigned to the label. The expression on the top of the stack remains there after creating the entry.

A right brace (}) symbolizes the end of the combinator file. There must be exactly one node on the stack, which is then returned.

This is a snippet of the combinator code – a function that accepts x as an argument and returns the x -th Fibonacci number:

```
C S _26 _30 @#0 @@C S _26 _30 @#1 @S' _100 _96 @B _394 @
C _192 _96 @#1 @B _394 @C _192 _96 @#2 @@@@#1 @@@#1 @:394
```

It is the compiled code of this function in Haskell:

```
fib :: Int -> Int
fib 0 = 1
fib 1 = 1
fib n = fib (n - 1) + fib (n - 2)
```

Notice the various types of atoms – the combinators (**S'** is a combinator name too), the integer literals such as #0, references starting with “_” and definitions starting with “:”.

When parsing this expression, the parser pushes C and then S onto the stack, followed by references to shared expressions 26 and 30. These expressions may not have been defined yet – in that case, the parser pushes a node that references the not-yet-defined expression. Otherwise, the referenced expression is found in the lookup table and is pushed onto the stack.

Then, upon encountering “@”, the parser pops $_30$ and $_26$ ¹² from the stack and pushes the application ($_26 _30$) onto the stack.

The parser continues similarly until the end, where only one node remains on the stack – it is the topmost application node of the function, to which the definition $:394$ gives a label in the shared expression table. Observe that in the body of the function, there is the reference $_394$ – this is a recursive call of the function.

The actual compiled program is also just one expression – it uses the **A** combinator to create a chained structure such as:

¹²Or the expressions with these labels, if they already have been defined and resolved.

(**A** (subexpression :label) next)

where ‘next’ can be another such application with **A** or it is the expression the program is supposed to be reduced to.

This makes the ‘subexpression’ a shared expression with a given label, but the expression reduces to ‘next’ as the reduction rule for **A** is $\mathbf{A} \ x \ y = y$. A chain of such applications with **A** combinators is used to define all the required shared expressions, and at the end of the file is the call to the “main” function of the program, which all the **A** combinator applications will be reduced to.

Combinator file example As an example, we will write the combinator code for this program by hand:

```
f :: Int -> Int
f x = 3 * x

g :: Int -> Int
g x = f (f x)

main = print (g 8)
```

The function **f**, in lambda calculus: $(\lambda x. * \ 3 \ x)$, is compiled to **(C * 3)** according to conversion and optimization rules from Sections 1.4.1 and 1.4.2. The function **g**, $(\lambda x. f \ (f \ x))$, is compiled to **(B f f)**. In postfix notation, that amounts to:

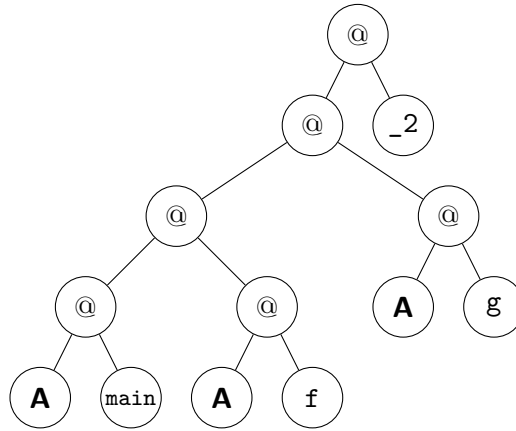
```
f = C * @ #3 @
g = B f @ f @
```

In a .comb file, each of these functions is assigned a numeric label. Therefore, the call to **f** in **g** is replaced with **_1** in our example. This is the (hand-written) combinator file of our example program, with a header and a closing brace at the end:

```
v0.1
3
A IO.print IO.stdout @ _0 #8 @@ :2 @
A C * @ #3 @ :1 @
A B _1 @ _1 @ :0 @
_2 @@@ }
```

The main function is defined on the first line of the body, it is assigned the label **:2** and then referenced by this number at the end of the file.

When parsing the file, the parser constructs the graph of this program, which looks like this (except the references such as **_2** might be already replaced by pointers to their respective nodes):



where

- `main` is the top node of the graph representation of `(IO.print IO.stdout (_0 #8))`, and as in the combinator file, we refer to `main` using label `_2`
- `f` is the top node of `(C * #3)` and is referred to using label `_1`
- `g` is the top node of `(B _1 _1)` and is referred to using label `_0`

While parsing, labels are also assigned to subexpressions. The runtime then begins with reducing the **A** combinator expressions, until the graph is reduced to `_2`. This label refers to the `main` function, and so the `main` function is run.

The MicroHs runtime would not be able to run this code because it needs the output handle (`stdout`) to be defined in a more complex way. Our runtime, however, can run the program, as it ignores the first argument to `IO.print`, evaluates the second argument, and outputs it to the screen. A description of the I/O execution is given in Section 3.3.4 and the specific I/O primitives are listed in Section 3.3.5.

2.5 The MicroHs C runtime

The MicroHs compiler uses a runtime system written in C, including a parser for the combinator files, a garbage collector, and, of course, an implementation of graph reduction and input/output handling.

When compiling for the default C runtime, MicroHs stores the content of the combinator file as a large C-style array of characters at the beginning of a C file. The file also includes required information about FFI calls used in the program and other metadata. This C file then gets compiled and linked with the MicroHs runtime to produce an executable.

Implementation

3.1 Programming languages and tools used

JavaScript (ECMAScript) [22] is the most widely used client-side web programming language [23]. TypeScript is a language that transpiles to JavaScript. Unlike JavaScript, which is a dynamically typed programming language, TypeScript has built-in type checking, which makes programs less prone to programmer mistakes and also improves integration with the editor [24]. Transpilation is the process of converting a TypeScript program to a JavaScript program – essentially this just removes the type annotations in the TypeScript program. Thus, the programmer can write (and debug) the code in TypeScript, but the code actually being run is in JavaScript, which is compatible with every major web browser [25] and other environments, such as Node.js [26].

3.2 About the codebase

We provide a new runtime system for MicroHs, implemented as a TypeScript [24] project, currently transpilable to JavaScript by just running the TypeScript compiler (`tsc`). The essential part of the project is the *Evaluator* class, which holds the evaluation logic and also allows for dependency injection. The functions `evaluate`, which evaluates a graph expression to WHNF (Section 1.2.2), and `execio`, which executes the Haskell/MicroHs monad I/O operations, are methods of the `Evaluator` class, in order to have access to its state and dependencies.

Although the program is currently made to be used as a console application using the Node.js [26] JavaScript runtime, the `Evaluator` class can also be used as a part of another application. For this, the `Output` and `Input` TypeScript interfaces have to be overridden, and objects of these concrete classes have to be passed to the constructor of the `Evaluator` class. For example, to use the program as a part of a website, a user can make an `Input` class that reads the

input from an HTML input field, and an `Output` class that writes the program output into another HTML text field.

3.3 Implementation details

3.3.1 Lexing and parsing

A lexer and parser were created to parse the MicroHs combinator output. The lexer is implemented as a simple deterministic finite automaton (or a state machine). The parser is a simple stack-based one, naively implemented to parse the MicroHs combinator file format, as described in Section 2.4. While parsing, the parser creates graph node objects (see next section) by calling corresponding functions, pushes the nodes onto and pops them from the stack as required.

3.3.2 Representation of the graph

The nodes of the graph are represented as JS objects, using TypeScript to create strict types. The general type of a graph node object is `GraphN`, which is a union type of `App` (application), `Const` (union of the constant/value types) or a `Pointer`. The constant types are `Comb` (combinator), `Str` (string), `Int` (integer), `FuncRef` (a function call/reference), and `Arr` (array). The exact definitions of some of these data types are listed in Code Listing 1.

The `Nodeptr` type is a kind of a wrapper for `GraphN`, we use it to emulate C-style pointers. Nodes such as `App` contain a `Nodeptr` for left-hand and right-hand side each, and so it is possible to rewrite the graph in-place. Additionally, the `Nodeptr` objects can be shared, which is used in the `Pointer` type. The `Pointer` type is used for sharing – multiple pointers point to a single `Nodeptr` object, which holds the shared expression graph.

There are also functions simplifying the code creating these objects, such as: `function combinator(name: string): Comb`.

Data types used As JavaScript has no built-in fixed-width integer type, we use the ES2020 `BigInt` [22] type for numbers. Each arithmetic operation calls the `BigInt.asIntN` or `BigInt.asUintN` function on the result. These functions truncate the result to the given width – in our case, that is 64 bits, which lets us emulate 64-bit wide integers. As MicroHs combinator output distinguishes between signed and unsigned arithmetic operations, operations with unsigned numbers are supported as well.

As in MicroHs, we use the combinators **A** and **K** for boolean values true and false, respectively. This is inverse to the Church encoding [2, pp. 58–59] in lambda calculus, where $TRUE := (\lambda t.(\lambda f.t))$, which would translate to the **K** combinator.

```

type GraphN = App | Const | Pointer

type Const = Comb | Str | Int | FuncRef | Arr

type App = {
  type: 'app';
  lhs: Nodeptr;
  rhs: Nodeptr;
}
type Pointer = {
  type: 'ptr';
  value: Nodeptr;
}
type Arr = {
  type: 'arr';
  array: Nodeptr[];
}

type Nodeptr = {
  n: GraphN
}

```

■ **Code listing 1** The types of graph node objects

The Arr and Str objects are wrappers for JS built-in strings and arrays. The functions in the compiled program require strings to be converted to their Cons-Nil representation, which is done by calling the `fromUTF8` built-in primitive function. The combinators **O** and **K** denote Cons and Nil, respectively. A Cons cell (`Cons a b`) – an ordered pair of `a` and `b` – is represented as (**O** `a b`) in both MicroHs and this runtime. Nil, or an empty list, is represented by the combinator **K**. The Cons and Nil combinators are used to create linked lists of integers (graph nodes of the `Int` type), each of which represents an ASCII value of a character in the string.

Both the Comb and FuncRef types contain a string representing the name of the combinator or function.

3.3.3 Expression evaluation

Our runtime system has two essential functions – `evaluate` and `execio`. The first one evaluates purely functional combinator expressions without side effects, with `unsafePerformIO` calls being the only exception, returning the expression reduced to WHNF. It implements the evaluation algorithm described in Section 1.5. The code of the `evaluate` function is listed in Code Listing 2.


```

evaluate(np: Nodeptr): GraphN {

  const lhs_stack: App[] = []
  let top = np.n
  let loopAgain = true

  while (loopAgain) {
    switch (top.type) {
      case 'app':
        lhs_stack.push(top)
        top = top.lhs.n
        break;
      case 'ptr':
        this.evaluate(top.value)
        top = top.value.n
        break;
      case "comb":
        [top, loopAgain] = evalCombExpr(top, lhs_stack)
        break
      case "funcref":
        [top, loopAgain] = this.evalFuncExpr(top, lhs_stack)
        // note: this is a reference to the Evaluator object
        break
      default:
        loopAgain = false
    }
    if (lhs_stack.length >= 1)
      lhs_stack[lhs_stack.length - 1].lhs.n = top
    np.n = lhs_stack[0] ?? top
  }

  if (lhs_stack.length > 0)
    top = lhs_stack[0]
  np.n = top
  return top;
}

```

■ **Code listing 2** The evaluate function code

The `evaluate` function takes the argument `np` (short for `Nodeptr`). This `Nodeptr` holds the root of the graph expression to evaluate. A local variable is used for the stack of ancestors. To make the code more concise, the node on the top of the stack is actually held separate from the rest of the stack, in the local variable `top`.

The `evaluate` function is implemented as a switch statement in a loop: if the current node on the top of the stack is a combinator, it performs the correct reduction step. If it is a call to a built-in function, the function is called after optional evaluation of its arguments via a recursive call to `evaluate`. If the node at the top is a `Pointer`, it first calls `evaluate` recursively and then puts the shared expression root on the stack. Finally, if the node on the top is an application, the `evaluate` function pushes the left subtree onto the stack.

After each reduction step, the node on the top of the stack, which is the result of the reduction, must be “connected” to the rest of the graph to ensure laziness. This is achieved by setting the top node as the left-hand side of the previous node on the stack, which is an application. If the top is the only node on the stack, the argument `np`, which is a `Nodeptr`, gets updated with the top node instead. This way, other occurrences of this expression will be updated with the (at least partially) evaluated expression.

The loop stops when the expression is in WHNF, the argument `np` is updated with this expression, and the expression is then returned by `evaluate`.

The `evaluate` function depends on the `evalCombExpr` and `evalFuncExpr` functions. These are called when the node on the top of the stack is a combinator or a built-in function reference, respectively. Both of them take the node on the top of the stack and the rest of the stack as arguments and are implemented as switch statements. When a definition with a name matching the combinator/function on the top is found, the required number of arguments is popped from the stack and the reduction is performed or the function is called. The result of the reduction / function call is returned, along with a boolean value, indicating whether the reduction / function call was performed successfully. When performing reductions of combinator expressions, it is important to copy the existing `Nodeptr` objects from the applications on the stack into the newly created applications. As an example, code that performs the reduction for the **S** combinator is shown in Code Listing 3. In the actual program, the construction of `App` nodes uses functions that make the code shorter.

If the required number of arguments is not available on the stack for the reduction / function call, the top node is returned unchanged, along with a false boolean value. This can happen e.g. when `evaluate` is called recursively and should not result in an error. A false value is also returned when the combinator/function on the top is an `IO` monad combinator or an `I/O` primitive – these are to be handled separately by `execio`. A reference to a combinator/function of an unknown name, on the other hand, makes `evalCombExpr` or `evalFuncExpr` raise an exception – this is useful for debugging and does not create a problem, as the runtime system does not support FFI.

```

case "S":
if (lhs_stack.length < 3) { return [top, false] }
else {
  const f: Nodeptr = lhs_stack.pop()!.rhs
  const g: Nodeptr = lhs_stack.pop()!.rhs
  const x: Nodeptr = lhs_stack.pop()!.rhs

  const lhs: Nodeptr = { n: { type: 'app', lhs: f, rhs: x } }
  const rhs: Nodeptr = { n: { type: 'app', lhs: g, rhs: x } }
  return [{ type: 'app', lhs, rhs } as App, true]
}

```

■ **Code listing 3** Code implementing the reduction rule for the **S** combinator

As is the case with MicroHs, this runtime does not implement any tail call optimization.

3.3.4 Input/output execution

The `execio`¹³ function executes the Haskell/MicroHs monad I/O operations. It is implemented to mimic what the MicroHs runtime does, just in a simpler way, because we do not have to manage memory allocation by hand.

The `IO` type is a monad which means it supports the operations ‘return’ and ‘bind’ (`>>=`). The operation ‘then’ (`>>`) is also used and it is represented using ‘bind’. [27, pp. 29–30] These operations are necessary when writing any effectful program in Haskell, which is a purely functional programming language, and, therefore, functions not outputting an `IO` type are without side effects.

The type annotations of these operations are as follows:

```

return :: b -> IO b
(>>=)  :: IO a -> (a -> IO b) -> IO b
(>>)   :: IO a -> IO b -> IO b

```

The ‘return’ operation returns an impure (i.e. with side effects) program (`IO b`) returning a value of type `b`. The ‘bind’ and ‘then’ operations are used for sequencing I/O operations. ‘Bind’ takes two I/O operations as arguments, executes the first, passes its result to the second, executes the second one, and returns its result. The ‘then’ operation just executes the first and the second I/O operation, returning the result of the second.

In the compiled program, we represent the `IO` monad with combinators `IO.>>=`, `IO.>>`, and `IO.return`. These, however, have no reduction rules – instead, they just represent an operation with the stack that `execio` will perform.

¹³Named after the function with the same purpose in the default MicroHs runtime.

An expression such as `IO.>> x y` gets converted to `(IO.>>= (K x) y)` – these are equal, because the **K** combinator will eliminate the argument passed to `(K x)`.

The only other type of allowed I/O expressions are I/O primitives. These include output, input, and manipulation with arrays. These are described in more detail in Section 3.3.5. Although `IO.return` is not an I/O primitive per se, it is included in the `execPrimitiveIO` function to keep the number of special cases handled in the code to only two: ‘bind’ and ‘then’.

The MicroHs `execio` function transforms the graph and keeps the expressions to execute later in this graph, using the ‘bind’ (`IO.>>=`) and special `C'BIND` combinators as type constructors to create a linked structure. Our implementation takes a simpler approach, using a single stack (named ‘continuation’) to store these expressions.

Exception handling The runtime system supports raising exceptions. This is not implemented in the `execio` function, instead, `raise` is a built-in function in `evalFuncExpr`. The `raise` primitive function evaluates its argument to obtain the exception message and throws a JavaScript exception named `ProgramRaisedException` with this message. Catching and handling exceptions is not supported.

3.3.5 Primitive functions implemented

To run programs compiled by MicroHs, our runtime system implements many primitive functions required by MicroHs.

These built-in functions are implemented in the `evalFuncExpr` function:

- Arithmetic functions `+`, `-`, `subtract` (`-` with reverse order), `*`, `quot` – integer division, `rem` – integer modulo, and unary negation `neg` (not to be confused with bitwise inversion). All of these have an unsigned `u` variant, 64-bit¹⁴ `I` variant and an unsigned 64-bit `Iu` variant. For example, supported addition operations are `+`, `u+`, `I+`, and `Iu+`.
- Simple comparison operations: `==`, `/=`, `<=`, `<`, `>=`, and `>` in 4 variants, similarly to integer operations.
- Three-way comparison: `cmp`, `icmp`, `Iicmp`, `ucmp`, and `Iucmp`. These follow a convention of MicroHs, returning `(Z K)` for lower-than, `(K K)` for equality, and `(K A)` for greater-than.
- Bitwise operations `and`, `or`, `xor`, `shl` (left shift), `shr` (right shift), `ashr` (arithmetic right shift), and `inv` (inversion). All of these also have a 64-bit variant with the prefix `I`.

¹⁴In our runtime, all operations are implemented as 64-bit, so the `I` and non-`I` variants are always the same.

- The `seq` operation, which evaluates both its arguments before returning the second. `seq` is used to improve performance by forcing strict evaluation (e.g. to save memory). [28, Section 6.2]
- `IO.performIO`, which calls `execIO` on its argument. A call of Haskell’s `unsafePerformIO` is translated to this function by `MicroHs`.
- `fromUTF8`, which takes a string as an argument and converts it to its `Cons-
Nil` representation.
- `raise`, which raises an exception.
- `equal` and `sequal`, universal equality operators supporting all types of expressions, required by older `MicroHs` versions.
- The following functions work as identity function, returning their only argument unchanged: `ord`, `chr`, `Itoi`, `itoI`, `utoU`, `UtoU`.

These I/O primitive functions are implemented in the `execPrimitiveIO` function:

- `putb` prints a character to the output. It takes two arguments: the ASCII value of a character to output and the output stream, which our runtime ignores.
- `getb` reads a character from the input and returns its ASCII value. It takes an argument – the input stream – which our runtime ignores.
- Array operations `A.alloc`, `A.read`, `A.size`, and `A.write`. In our runtime, these manipulate JavaScript-style arrays and are required to run even the most basic programs, since `MicroHs` uses them internally when doing any I/O.
- `IO.print` takes two arguments: the output stream, which we ignore, and the graph of the expression to dump. It prints the string representation of the given expression to the output. `IO.print` is not required for running programs with basic I/O and it is supported only to run some test programs, for example, see our handwritten program in Section 2.4.

3.4 Measurements

To compare our runtime to other options for compiling Haskell to JS/Wasm, we use the sample programs `FibRec.hs`, which calculates the 31th number in the Fibonacci sequence naively using recursion, and `Primes.hs`, which is a Haskell implementation of the sieve of Eratosthenes to calculate the first 500 prime numbers [29, pp. 222–223]. These programs can be found in the `runtime/samples/measure` directory of the attachment.

We compiled these programs to the MicroHs combinator code, to use with our runtime, with the command:

```
mhs <Name>.hs -o <name>.comb
```

We have also used MicroHs’s `emscripten` target to compile the program to JavaScript/WebAssembly using Emscripten [30]. This creates a single JS file, where the Haskell program is bundled with the default MicroHs runtime. We used MicroHs version 0.14.27.0 and Emscripten version 5.0.3. We compiled the programs using the command:

```
mhs <Name>.hs -temscripten -o <Name>.js
```

Another option to compile Haskell programs to JavaScript is the JS backend of GHC [20], which also uses Emscripten, and it creates a large JS file with an accompanying folder of auxiliary files. This compiler can be installed using the guide from [31]. We used the version 9.6.7 of GHC. This compilation method is called using the command:

```
javascript-unknown-ghcjs-ghc -main-is <Name> <Name>.hs
```

As a reference, we also compiled the programs to a native executable using MicroHs with the command:

```
mhs <Name>.hs -o <name>.out
```

The size of our runtime transpiled to JavaScript is 72 KiB¹⁵. For a fair comparison with the other options, we should also include the size of the combinator file, as the other compilers produce a file where the runtime and the compiled program are bundled. The sizes of the compiled file are compared in Table 3.1.

Compiler/Runtime	FibRec.hs	PrimesT.hs
Our runtime + .comb file	108 KiB	110 KiB
MicroHS & Emscripten	415 KiB	424 KiB
GHC JS backend	6.6 MiB	7.7 MiB
MicroHs native	233 KiB	241 KiB

■ **Table 3.1** Comparison of compiled program size

We have also measured the duration of the program running and the memory usage, using the `cgmementime` program [32]. We used the `group_mem_high` metric reported by this program, which is the sum of memory used by all the child processes created by the measured command. Node.js [26] version 22.22.2 was used to run the JS/Wasm files. The measurements were made on a notebook with an AMD Ryzen 7 PRO 6850U processor.

Tables 3.2 and 3.3 present the measured execution time and memory usage per program, respectively.

Compiler/Runtime	FibRec.hs	PrimesT.hs
Our runtime	2.215 s	3.676 s
MicroHS & Emscripten	1.390 s	1.507 s
GHC JS backend	0.763 s	0.344 s
MicroHs native	0.335 s	0.501 s

■ **Table 3.2** Measured execution times of the sample programs

Compiler/Runtime	FibRec.hs	PrimesT.hs
Our Runtime	37,180 KiB	102,580 KiB
MicroHS & Emscripten	324,308 KiB	485,292 KiB
GHC JS backend	41,384 KiB	53,840 KiB
MicroHs native	383,220 KiB	606,192 KiB

■ **Table 3.3** Measured memory usage of the sample programs

The compiled files can be found in the `runtime/samples/outputs` folder of the attached archive. The combinator files have the `.comb` extension, the files created by MicroHs with Emscripten have the `.js` extension. GHC with the JS backend creates a file with no extension (e.g. `FibRec`) and a folder of the same name with suffix `.jsexec`. The native executables created by MicroHs have the `.out` extension.

3.5 Testing the functionality

The runtime has been tested with a set of simple Haskell programs, testing features such as I/O, recursion, algebraic data types and printing them. Special attention was paid to testing the correct implementation of laziness. More details about the tests, including how to run them, can be found in Section A.3.

3.6 Current limitations

Printing unsigned numbers larger than $2^{63} - 1$ is not working correctly; instead, they are interpreted as signed, i.e., $2^{64} - 1$ is printed as -1 . Additionally, some Haskell/MicroHs features are not supported, such as `rnf`, which forces evaluation of an expression to its normal form [33].

¹⁵The source mapping files (generated by the transpiler for debugging) and the lambda expression evaluator (Section A.4) are not included.

Conclusion

The goal of this work was to implement a runtime system for the MicroHs compiler, using the TypeScript programming language. With the resulting runtime system supporting the basic operations, including arithmetic operations on 64-bit integers and console input and output, all requirements are fulfilled.

The most challenging part of the work was getting the I/O operations from the MicroHs compiled program to run, because the combinator representation of a program is somewhat cryptic and, therefore, it is hard to debug the evaluation and execution process.

The result is a console application, easy to adapt for use as a component in other projects. The application has been thoroughly tested, is simple, extendable, and has a small footprint, while being reasonably quick.

Future work might be aimed at adding support for more data types, such as floats, adding support for FFI (foreign function interface), generating a single file that bundles the runtime with a graph representation of a program, and optimizing the runtime for better speed, with the ultimate goal of being able to run the MicroHs compiler itself.

Appendix A

Usage of the application

The supported operating system for our runtime system is Linux.

A.1 Using MicroHs to compile Haskell to combinator code

To use our runtime for running Haskell programs, it is required to compile them into combinator (`.comb`) files using MicroHs.

The recommended version of MicroHs is `v0.14.27.0`, with git commit hash:

```
7120b95c11023fea1d5b82ed7a10e2f5709b93ac
```

Namely, the most recent versions released during the development of our runtime systems have added new combinators for tuples, which are not supported.

To install MicroHs on Linux, use these commands:

```
git clone https://github.com/augustss/MicroHs.git
cd MicroHs
git checkout 7120b95c11023fea1d5b82ed7a10e2f5709b93ac
make install
export PATH="$PATH:/home/$USER/.mcabal/bin"
# optionally: rm -rf MicroHs
```

These commands clone the source code from the MicroHs GitHub repository [17], check out the recommended version, and install it. The installation script installs MicroHs to the user's home directory, no superuser rights are required. The `export` command adds the MicroHs installation folder to `PATH`, this command should also be added to the user's shell configuration script (e.g. `.bashrc`).

Use the following command to compile a Haskell program named `Program` into a combinator file:

```
mhs Program.hs -o program.comb
```

The Haskell file should begin with `module <name> where` and the file name should then be `<name>.hs`. The combinator file must have the `.comb` extension.

A.2 Usage of the runtime

The runtime requires the Node.js [26] runtime and the `npm` package manager [34] to be installed.

The dependencies, namely the TypeScript compiler `tsc` [24] can be installed by running

```
npm install
```

in the `runtime` folder of the attached archive. The runtime is compiled by running

```
npm run build
```

or alternatively

```
npx tsc
```

. The output of the compilation goes into the `dist` directory.

A combinator file named `program.comb` can then be run with the command:

```
node dist/main.js program.comb
```

The attached archive contains the output of the compilation, so that only Node.js is required to run. To clean the compiled output, remove the `dist` directory.

A.3 Running the tests

The test files are included in the attached archive, in the `runtime/tests` directory. Other interesting programs, for example the programs used for measurements in Section 3.4, are included in `runtime/samples`¹⁶.

Before running the tests, the runtime must be compiled by following the instructions in the previous section.

¹⁶The samples also include the program `InFib.hs`, which loads an integer x from the input and calculates x^{th} number in the Fibonacci sequence, indexed from 0. This can be also used to test exceptions, as an exception is raised when an integer cannot be parsed.

The tests are run using the Bash script `test.sh`. This script first creates a combinator file for each Haskell program in the `tests` directory, then uses MicroHs or GHC (depending on the `REFHS` variable set in the script) to create reference outputs. It uses timestamps to do this only when the combinator file or reference output is nonexistent, older than the Haskell program, or older than the test script itself.

It then tests the runtime with the compiled combinator files, comparing its output to the reference output. If any of the tests fail, status code of 1 is returned at the end.

The Haskell files must have the `.hs` suffix and no spaces in the names. The name before the suffix must be the same as the module name and must start with an uppercase letter. If there is a file with the same name and the suffix `_in.txt`, this file is piped into the Haskell program while running – this allows for testing programs with input.

```
# to ensure that all test cases are freshly
# compiled, run:
rm -f tests/combinators/* tests/*_out.txt
# to run the tests, run:
./test.sh
# to skip the compilation of combinator files
# and creation of reference outputs, run:
TEST_ONLY=1 ./test.sh
```

The attached archive contains the compiled combinator files and reference outputs, so the creation of test cases can be skipped.

The most interesting test programs are the tests of laziness:

- **Lazy1.hs** reads the first 12 items in an infinite list of ones. This requires laziness, as an eager (non-lazy) evaluator would try to evaluate the rest of the list, getting stuck in a loop.
- **Lazy2.hs** tests correct sharing. It prints the sum of the first three items in a list twice. The list consists of `(sideEffect n)` cells for `n` from 10 to 1. A call to `(sideEffect n)` prints "side effect" and returns `n`.

If sharing is implemented correctly, the list will be shared and thus the `sideEffect` function will be called only three times. Without sharing, each of the first three items of the list would get evaluated twice, leading to the string "side effect" being printed six times in total.

- **Lazy3.hs** includes more similar tests of laziness, including a test that combines sharing and local recursion.

Another interesting test program is `PPA2025.hs`, which tests the solutions to some tasks in the PPA course at the FIT CTU.

A.4 Lambda expression evaluator

Included in the `runtime` folder is also an evaluator for lambda calculus, used mainly for debugging in the early stages of development. It parses a lambda expression from its standard output, compiles it to SKI using the optimizations described by Peyton Jones (Sections 1.4.1 and 1.4.2) and then evaluates it. The evaluator code is taken from the runtime, the largest difference is that the representation of boolean values was inverted to match the Church encoding¹⁷ [2, pp. 58–59]. Also, unnecessary code was removed and a simple function with a side effect was added, to be able to showcase lazy evaluation.

The lambda calculus evaluator is run using the following command:

```
node dist/lambda/main.js
```

It reads the lambda expression from the standard input, in the Bash shell, the expression can be entered using the “here string” syntax:

```
node dist/lambda/main.js <<< '(\x. * 2 x) 5'
```

Another example of a supported lambda expression is

```
Y (\f. (\x . = x 0 1 (* (f (- x 1)) x) ) ) 6
```

which calculates the factorial of 6.

The evaluator supports the backslash (`\`) or the U+03BB `λ` character as the lambda symbol in abstractions. Spaces are required between variables and functions.

The lambda calculus evaluator supports numbers and the following operations and functions:

- the **Y** combinator, which is represented directly by the **Y** combinator¹⁸
- arithmetic operations `+`, `-`, `*`, `/`
- comparisons `=`, `/=`, `<`, `<=`, `>`, `>=`
- the function `printAndReturn`, which evaluates its argument, prints the argument to the output, and returns it

The script `lambda_test.sh` runs the tests in the `lambda_tests` folder. The evaluator first prints the parsed expression and the expression after conversion to SKI combinators. These are ignored, the test script compares the rest of the output, which is the console output of the program (`printAndReturn` function) and the resulting expression in WHNF with the reference output.

¹⁷Although the compilation does not produce code with the **A** combinator, it is used as the Church-encoded false value.

¹⁸The measured speed improvement versus using the lambda calculus-style **Y** combinator was very insignificant.

Bibliography

1. BARENDREGT, H. P. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland, 1984. Studies in logic and the foundations of mathematics. ISBN 0444875085.
2. PIERCE, Benjamin C. *Types and programming languages*. Cambridge, Mass: MIT Press, 2002. ISBN 0262162091, ISBN 9780262162098.
3. MAZZOLA, G.; MILMEISTER, G.; WEISSMANN, J. *Comprehensive Mathematics for Computer Scientists 2: Calculus and ODEs, Splines, Probability, Fourier and Wavelet Theory, Fractals and Neural Networks, Categories and Lambda Calculus*. Springer Berlin Heidelberg, 2005. Universitext. ISBN 9783540269373. Available also from: <https://books.google.cz/books?id=FatbNhzs3YMC>.
4. SESTOFT, Peter. Demonstrating Lambda Calculus Reduction. In: *The Essence of Computation: Complexity, Analysis, Transformation*. Ed. by MOGENSEN, Torben Æ.; SCHMIDT, David A.; SUDBOROUGH, I. Hal. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 420–435. ISBN 978-3-540-36377-4. Available from DOI: 10.1007/3-540-36377-7_19.
5. PEYTON JONES, Simon L. *The implementation of functional programming languages*. Prentice-Hall, Inc., 1987. Prentice-Hall International Series in Computer Science. Available also from: <https://simon.peytonjones.org/assets/pdfs/slpj-book-1987-2up-searchable.pdf>.
6. HASKELL.ORG, INC. *Haskell* [comp. software]. [visited on 2026-04-17]. Available from <https://www.haskell.org/>.
7. WADSWORTH, Christopher Peter. *Semantics and pragmatics of the lambda-calculus*. 1971. PhD thesis. University of Oxford, Programming Research Group.

8. HENDERSON, Peter; MORRIS, James H. A lazy evaluator. In: *Proceedings of the 3rd ACM SIGACT-SIGPLAN Symposium on Principles on Programming Languages*. Atlanta, Georgia: Association for Computing Machinery, 1976, pp. 95–103. POPL '76. ISBN 9781450374774. Available from DOI: 10.1145/800168.811543.
9. TURNER, David A. A new implementation technique for applicative languages. *Software: Practice and Experience*. 1979, vol. 9, no. 1, pp. 31–49. Available from DOI: <https://doi.org/10.1002/spe.4380090105>.
10. TURNER, D. An overview of Miranda. *ACM SIGPLAN Notices*. 1986, vol. 21, no. 12, pp. 158–166. ISSN 0362-1340. Available from DOI: 10.1145/15042.15053.
11. HUDAK, P.; HUGHES, J.; PEYTON JONES, S. L.; WADLER, P. A history of Haskell: being lazy with class. In: *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*. Association for Computing Machinery, 2007. ISBN 9781595937667. Available from DOI: 10.1145/1238844.1238856.
12. SCHÖNFINKEL, Moses. Über die Bausteine der mathematischen Logik. *Mathematische Annalen*. 1924, vol. 92, pp. 305–316. Available also from: <https://api.semanticscholar.org/CorpusID:118507515>.
13. CURRY, Haskell B. Grundlagen der Kombinatorischen Logik. *American Journal of Mathematics* [online]. 1930, vol. 52, no. 3, pp. 509–536 [visited on 2026-03-11]. ISSN 00029327, ISSN 10806377. Available from: <http://www.jstor.org/stable/2370619>.
14. CHURCH, Alonzo. A set of postulates for the foundation of logic. *Annals of mathematics*. 1932, vol. 33, no. 2, pp. 346–366.
15. TURNER, David A. Another algorithm for bracket abstraction. *The Journal of symbolic logic*. 1979, vol. 44, no. 2, pp. 267–270. ISSN 0022-4812.
16. AUGUSTSSON, Lennart. MicroHs: A Small Compiler for Haskell. In: *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium*. Milan, Italy: Association for Computing Machinery, 2024, pp. 120–124. Haskell 2024. ISBN 9798400711022. Available from DOI: 10.1145/3677999.3678280.
17. AUGUSTSSON, Lennart. *MicroHs (Micro Haskell)* [comp. software]. [visited on 2026-04-17]. Available from <https://github.com/augustss/MicroHs>.
18. PEYTON JONES, S. L.; HALL, Cordy; HAMMOND, Kevin; PARTAIN, Will; WADLER, Philip. The Glasgow Haskell compiler: a technical overview. In: *Proc. UK Joint Framework for Information Technology (JFIT) Technical Conference*. 1993, vol. 93, p. 13.

19. FAUSAK, Taylor; HASKELL WEEKLY. *2017 state of Haskell survey results* [online]. [visited on 2026-04-24]. Available from: <https://taylor.fausak.me/2017/11/15/2017-state-of-haskell-survey-results/#question-14>.
20. IOG ENGINEERING; HENRY, Sylvain, et al. *JavaScript backend merged into GHC* [online]. [visited on 2026-04-23]. Available from: <https://web.archive.org/web/20250723163008/https://engineering.iog.io/2022-12-13-ghc-js-backend-merged/>. Archived from <https://engineering.iog.io/2022-12-13-ghc-js-backend-merged/> on 2025-07-23.
21. GHCJS. *GHCJS* [comp. software]. [visited on 2026-04-16]. Available from <https://github.com/ghcjs/ghcjs>.
22. ECMA INTERNATIONAL. *ECMA-262: ECMAScript® 2020 Language Specification* [online]. 11th ed. Geneva, 2020 [visited on 2026-05-14]. Available from: <https://262.ecma-international.org/11.0/>.
23. W3TECHS; Q-SUCCESS. *Usage statistics of JavaScript as client-side programming language on websites*. W3Techs – Web Technology Surveys [online]. [visited on 2026-04-18]. Available from: <https://w3techs.com/technologies/details/cp-javascript>.
24. TYPESCRIPT; MICROSOFT CORPORATION. *TypeScript* [online]. [visited on 2026-04-18]. Available from: <https://www.typescriptlang.org/>.
25. DEVERIA, Alexis. *Can I Use — ECMAScript 2015*. Can I Use? [online]. [visited on 2026-04-18]. Available from: <https://caniuse.com/es6>.
26. OPENJS FOUNDATION. *Node.js* [online]. [visited on 2026-04-18]. Available from <https://nodejs.org/en>.
27. WADLER, Philip. Monads for functional programming. In: *Advanced Functional Programming*. Ed. by JEURING, Johan; MEIJER, Erik. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 24–52. ISBN 0302-9743.
28. PEYTON JONES, Simon. *Haskell 98 Language and Libraries The Revised Report*. Cambridge: Cambridge University Press, 2003. ISBN 9780521826143.
29. HUTTON, Graham. *Programming in Haskell*. 2nd ed. Cambridge University Press, 2016.
30. EMSRIPTEN CONTRIBUTORS. *Emscripten* [comp. software]. [visited on 2026-05-13]. Available from <https://emscripten.org/>.
31. ADHATRAO, Tushar. *Step-by-Step Guide to Installing GHC-JS (Haskell JavaScript FFI)* [online]. [visited on 2026-05-13]. Available from: https://www.tushar-adhatrao.in/blogs/haskell_javascript_ffi.html.

32. SAUTHOFF, Georg. *cgmvertime* [comp. software]. [visited on 2026-05-13]. Available from <https://github.com/gsauthof/cgmvertime>.
33. MARLOW, Simon; MAIER, Patrick; LOIDL, Hans-Wolfgang; ASWAD, Mustafa K; TRINDER, Phil. Seq no more: Better Strategies for Parallel Haskell. In: *Haskell '10: Proceedings of the Third ACM SIGPLAN Symposium on Haskell* [online]. Baltimore, USA: ACM, 2010 [visited on 2026-05-14]. Available from: <http://simonmar.github.io/bib/papers/strategies.pdf>.
34. NPM, INC. *Downloading and installing Node.js and npm* [online]. [visited on 2026-05-12]. Available from: <https://docs.npmjs.com/downloading-and-installing-node-js-and-npm>.

Contents of the attachment

```
/
├── runtime.....directory with the runtime implementation files
│   ├── dist.....directory with the compiled runtime
│   ├── lambda_tests.....directory with lambda expressions for tests
│   ├── samples.....directory with sample Haskell programs
│   │   └── outputs.....directory with compiled samples
│   ├── src.....directory with the runtime source code
│   ├── tests.....directory with test for the runtime
│   │   └── combinators.....directory with tests compiled to .comb files
│   ├── eslint.config.mts.....file with linter configuration
│   ├── lambda_test.sh.....script for testing the lambda evaluator
│   ├── LICENSE.....file with the license of the project
│   ├── package.json.....file specifying the required packages
│   ├── package-lock.json.....file specifying versions of the packages
│   ├── README.md.....a description of the project
│   ├── test.sh.....script for testing the runtime
│   ├── tsconfig.json.....file with TypeScript configuration
│   └── CONTENTS.txt.....description of the archive contents
├── thesis.....directory with the source code of the thesis
└── thesis.pdf.....the text of the thesis
```